

 Universidade do Porto Faculdade de Engenharia FEUP	Faculdade de Engenharia da Universidade do Porto Licenciatura em Engenharia Informática e Computação Programação em Lógica Exame em 11.01.2003	2002/2003 LEIC Duração: 2h30m
--	--	--

GRUPO I (8 val.)

1) Construa o predicado *remove_todos(+X, ?Lista, ?Nova)*, que suceda se *Nova* é a lista que resulta de *Lista* por remoção de todas as ocorrências do elemento *X*.

2) Construa um predicado *extremos(+Lista, -Maximo, -Minimo)* que dada uma lista, determine os seus valores máximo e mínimo.

Exemplo:

extremos([5, 10, 2, 30, 6], Max, Min)? tem como resultado *Max = 30* e *Min = 2*.

3) Suponha que possui um predicado Prolog descrevendo informação sobre cidades e vilas da Europa. Os factos têm o formato *europa(Nome, Pais, Tipo, Populacao)*, onde *Nome* é o nome da cidade/vila, *Pais* é o nome do país a que a cidade/vila pertence, *Tipo* é um argumento que pode ter três valores (capital, cidade ou vila) e *Populacao* é um inteiro que representa o número de habitantes (em milhares).

- Construa um predicado *idades_grandes(+Pais, -Lista, -Pop_Total)*, que determine a lista de cidades (exceptuando a capital do país) que têm população superior a 100 mil habitantes e a soma das suas populações.
- Construa um predicado *capitais(-Lista_Capitais)*, que determine uma lista com as capitais e número de cidades (incluindo a capital) de todos os países.

Exemplo:

europa(lisboa, portugal, capital, 900).

europa(porto, portugal, cidade, 300).

europa(madrid, espanha, capital, 3100).

europa(barcelona, espanha, cidade, 2900).

europa(espinho, portugal, cidade, 40).

europa(gaia, portugal, cidade, 260).

europa(arcozelo, portugal, vila, 15).

europa(coimbra, portugal, cidade, 105).

? cidades_grandes(portugal, L, P).

L = [(porto, 300), (gaia, 260), (coimbra, 105)]

P = 665

? capitais (L).

*L = [lisboa*6, madrid*2]*

GRUPO II (7 val.)

O jogo do solitário é jogado por um único jogador que em cada jogada pode “comer” uma peça horizontal ou verticalmente (jogadas na diagonal não são válidas), saltando para tal sobre uma outra peça (tal como nas damas). Uma peça pode “comer” outra peça, unicamente se existe uma casa livre a seguir à peça que vai “comer”. Para conseguir concluir o jogo, o tabuleiro deve ficar com uma única peça, localizada no centro do mesmo.

Suponha que se faz uma representação em Prolog do tabuleiro deste jogo usando uma lista em que cada elemento representa uma posição do tabuleiro (e cada posição do tabuleiro está representada na lista). O termo Prolog usado tem a forma *(Y, X, Peca)* em que *Y* e *X* correspondem respectivamente à linha e coluna e *Peca* pode ter os valores *o* (peça presente na casa *Y,X*) ou *x* (casa vazia). O centro do tabuleiro corresponde à casa *(Y,X)=(4,4)*. A configuração inicial do tabuleiro é representada por:

```
tabuleiro( [ (1,3,o), (1,4,o), (1,5,o), (2,3,o), (2,4,o), (2,5,o),
              (3,1,o), (3,2,o), (3,3,o), (3,4,o), (3,5,o), (3,6,o), (3,7,o),
              (4,1,o), (4,2,o), (4,3,o), (4,4,x), (4,5,o), (4,6,o), (4,7,o),
              (5,1,o), (5,2,o), (5,3,o), (5,4,o), (5,5,o), (5,6,o), (5,7,o),
              (6,3,o), (6,4,o), (6,5,o), (7,3,o), (7,4,o), (7,5,o) ] ).
```

1) Escreva um predicado *mostra_tab(+Tab)* que dado um tabuleiro de solitário representado como uma lista de elementos do tipo *(Lin, Col, Peca)*, escreva o tabuleiro no écran da seguinte forma:

```

      1 2 3 4 5 6 7
    # # # # # # #
1 # # # o o o # # # 1
2 # # # o o o # # # 2
3 # o o o o o o o # 3
4 # o o o . o o o # 4
5 # o o o o o o o # 5
6 # # # o o o # # # 6
7 # # # o o o # # # 7
    # # # # # # #
      1 2 3 4 5 6 7

```

2) Construa um predicado *vizinho(+Tab, ?Xi, ?Yi, ?Xf, ?Yf, ?Direc)* que suceda se a casa *Yf/Xf* for vizinha de *Yi/Xi* na direcção *Direc* que pode tomar os valores *esq*- esquerda, *dir* – direita, *cim* – cima e *bai* – baixo. Por exemplo:

```

:- tabuleiro(Tab), vizinho(Tab, 4, 4, Xf, Yf, dir).   deverá retornar Xf =5, Yf = 4.
:- tabuleiro(Tab), vizinho(Tab, Xi, Yi, 5, 4, dir).   deverá retornar Xf =4, Yf = 4.
:- tabuleiro(Tab), vizinho(Tab, 3, 3, 6, 4, cim).     deverá retornar false.

```

3) Suponha que dispõe dos predicados *jogada_valida(+Tab, ?Lini/Coli, ?Linf/Colf)* que determine se a jogada de *Lini/Coli* para *Linf/Colf* é válida num determinado tabuleiro *Tab* e *executa_jogada(+Tab, +Lini/Coli, +Linf/Colf, -NovoTab)* que calcule o tabuleiro resultante da execução de uma jogada num dado tabuleiro. Escreva um predicado *resolve_solitario(+Tab, -Lista_Jogadas)* que dado um determinado tabuleiro, resolve o problema utilizando pesquisa em profundidade com backtracking, retornando a lista de jogadas, na forma *(Xi/Yi-Xf/Yf)* necessárias para o resolver.

GRUPO III – Programação em Lógica com Restrições (5 val)

Suponha o problema da geração de horários de um curso numa escola. Existem 2 salas (S1 e S2), 4 professores (P1, P2, P3 e P4) e cada um lecciona uma das 4 disciplinas que os alunos têm de frequentar (D1, D2, D3 e D4, respectivamente). Os alunos estão agrupados em 2 turmas (T1, T2). As aulas são todas de 3 horas e podem começar às 8, 11, 14 ou 17 horas, de Segunda a Quinta-feira. Cada disciplina tem 1 aula semanal.

Pretende-se resolver o problema usando Programação em Lógica com Restrições em domínios finitos (CLP-FD).

1) Formule o problema como um problema de satisfação de restrições. Para esse efeito indique as variáveis escolhidas e as estruturas usadas para as agrupar. Tenha em atenção que terá de impor a estas variáveis as restrições intrínsecas do problema:

- Um docente não pode dar duas aulas ao mesmo tempo
- Uma turma não pode ter duas aulas ao mesmo tempo
- Uma sala não pode ter duas aulas ao mesmo tempo.

2) Escreva os predicados necessários para impôr as restrições anteriores na linguagem CLP-FD do Sicstus Prolog.

3) Complete o programa de geração de horários. (Não é necessário tratar a visualização).

4) Verificou-se que havia algumas condicionantes adicionais aos horários. Introduza as restrições necessárias para as garantir.

- As aulas da disciplina 1 não podem ser dadas na sala 1
- O docente P1 não dá aulas ao mesmo tempo que o docente P2
- O docente P1 está indisponível às segundas-feiras
- O docente P2 está indisponível das 8 às 11 da manhã
- A sala S2 não pode ser utilizada às 17 horas de Quarta e de Quinta-feira.